Learning Memory System

Condensed Study Notes

Generated: 2026-05-24 23:55 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION

Computer architecture and organization refer to the study of how computer systems are designed and how their components work together.

Architecture defines the functional behavior of a computer system, essentially its programmer's view. Organization describes the operational units and their interconnections that realize the architectural specifications.

COMPUTER MEMORY SYSTEM

The computer memory system is a hierarchy of storage devices, organized by speed and cost.

- **Registers**: Fastest, smallest, located within the CPU. Hold data the CPU is currently processing.
- **Cache Memory**: Faster than main memory, smaller. Stores frequently used data from main memory to reduce CPU access time.
- **Main Memory (RAM)**: Slower than cache, larger. Holds programs and data currently in use by the system.
- **Secondary Storage**: Slowest, largest, non-volatile. Stores data and programs long-term (e.g., Hard Drives, SSDs).

Learning Objectives

By the end of this module, you will be able to:

1. Understand the main characteristics of computer memory systems and the use of memory hierarchy.
2. Understand cache memory and the key elements of cache design.

Outline

This section will cover:

- **Cache Memory Principles**: Key aspects of cache design.
- **Elements of Cache Design**: Specific factors influencing cache performance.
- **Organization, DRAM and SRAM**: How memory is structured, focusing on Dynamic Random Access Memory (DRAM) and Static Random Access Memory (SRAM).
- **The Memory Hierarchy**: The layered structure of computer memory.
- **The Memory System**: An overview of how memory components work together.
- **Interleaved Memory**: A technique for improving memory access speed.
- **Memory Characteristics**: Properties that define memory performance and behavior.

- **Semiconductor Main Memory**: The nature of primary storage built from semiconductor technology.
- **Types of ROM**: Different forms of Read-Only Memory.
- **Chip Logic**: The internal circuitry of memory chips.
- **Chip Packaging**: How memory chips are housed and connected.

3.1 Memory Characteristics

Memory characteristics describe how data is stored and accessed.

1. Location: Where memory resides relative to the processor.

- **Internal:** Resides within or very close to the processor (e.g., registers, cache, main memory).
- **External:** Peripheral storage devices (e.g., disk, tape).

2. Capacity: The total amount of data a memory unit can store, typically measured in Bytes.

3. Unit of Transfer: The amount of data moved at one time.

- **Internal Memory:** Often matches the data bus width (e.g., 64, 128 bits).
- **External Memory:** Usually transferred in larger chunks called blocks.

4. Method of Accessing: The order in which data can be retrieved.

- **Sequential Access:** Data accessed in a linear order (e.g., tape). Access time varies.
- **Direct Access:** Access to blocks based on physical location, often involving sequential searching to reach the final spot. Access time varies.
- **Random Access:** Any location can be accessed directly, with constant access time regardless of previous accesses (e.g., main memory). Each location has a unique address.
- **Associative Access:** A type of random access where data is retrieved based on a portion of its content, not its address. All words are checked simultaneously for a match. Cache memories can use this.

5. Performance: Key metrics for how quickly data can be accessed and transferred.

- **Access Time (Latency):** The time to complete a read or write operation. For random access, it's constant. For sequential/direct, it's variable.
- **Memory Cycle Time:** For random access, it's access time plus any delay before the next access can start.
- **Transfer Rate (R):** The speed at which data moves into or out of memory, measured in bits per second (bps). For random access, R = 1 / (cycle time).

Computer Memory

Computer memory stores information for immediate use by a computer or digital device. It's a core component of the CPU.

Memory can be categorized by various characteristics, including:

- **Location**: Internal (e.g., processor registers, cache, main memory) or External (e.g., optical disks, magnetic disks, tapes).
- **Performance**: Access time, cycle time, transfer rate.
- **Physical Type**: Semiconductor, Magnetic, Optical, Magneto-optical.
- **Capacity**: Number of words, Number of bytes.
- **Unit of Transfer**: Word, Block.
- **Physical Characteristics**: Volatile/nonvolatile, Erasable/nonerasable.
- **Organization**: Sequential, Direct, Random, Associative.

Due to diverse digital device needs, a single memory technology is insufficient. Therefore, computer systems use a hierarchy of memory subsystems, with internal memories directly accessible and external memories accessed via I/O modules.

3.2 The Memory Hierarchy

Computer systems use a memory hierarchy to minimize access time. This organization is based on the principle of **locality of reference**, which states that a processor tends to access a clustered set of memory locations repeatedly over short periods.

There's a fundamental trade-off among memory's key characteristics: capacity, access time, and cost. Generally:

- Faster access time comes with greater cost per bit and smaller capacity.
- Greater capacity comes with slower access time and smaller cost per bit.

To balance these, smaller, faster, more expensive memories are supplemented by larger, cheaper, slower memories.

A typical memory hierarchy includes:

- **Registers**: Fastest, smallest, most expensive, internal to the processor.
- **Cache**: Higher-speed, smaller memory that stages data between main memory and registers to improve performance. Not usually visible to the programmer or processor.
- **Main Memory**: Principal internal memory system with unique addresses. Usually extended by cache.
- **External Mass Storage (Secondary/Auxiliary Memory)**: Slowest, largest, cheapest. Stores program and data files (e.g., hard disks, tapes, optical storage). Can be used to extend main memory via virtual memory.

Memories like registers, cache, and main memory are typically volatile and use semiconductor technology. External memory is nonvolatile.

3.3 Cache Memory

Cache memory acts as a high-speed buffer between the CPU and main memory, designed to reduce average data access time.

It combines the speed of expensive, small memory with the capacity of less expensive, slower memory. A cache stores copies of frequently accessed data and instructions from main memory.

When the CPU needs a word, it first checks the cache. If the word is present (a cache hit), it's delivered quickly. If not (a cache miss), a block of data from main memory containing that word is loaded into the cache, and then the word is delivered to the CPU. This process leverages the principle of locality of reference, anticipating that data fetched into the cache will be needed again soon.

Multiple levels of cache exist (L1, L2, L3), with each successive level being slower and larger than the previous one.

Key Terms:

- **Cache Hit**: When requested data is found in the cache.
- **Cache Miss**: When requested data is not found in the cache, requiring retrieval from main memory.
- **Block**: A fixed-size group of words in main memory.
- **Line**: A block of memory stored in the cache, consisting of K words plus a tag and control bits.
- **Tag**: Bits within a cache line that identify which specific block from main memory is currently stored there.
- **Line Size**: The number of words in a cache line (excluding tag and control bits).
- **Locality of Reference**: The principle that memory accesses tend to cluster in time and space, meaning recently accessed data or data near it is likely to be accessed again soon.

3.3.1 Cache Performance

Cache performance is primarily measured by the Hit Ratio. A **cache hit** occurs when the processor finds the requested memory location already in the cache. Conversely, a **cache miss** happens when the location is not found in the cache. Upon a miss, the cache fetches the data from main memory, stores it in a new cache entry, and then fulfills the processor's request.

To improve cache performance, several strategies can be employed:

- Increase the cache block size.
- Increase cache associativity.
- Reduce the miss rate (frequency of misses).
- Reduce the miss penalty (time taken to retrieve data on a miss).
- Reduce the time to achieve a cache hit.

3.3.2 Cache Read operation

When the processor needs to read data, it sends a read address (RA).

- **Cache Hit**: If the requested word is in the cache, it's immediately sent to the processor.
- **Cache Miss**: If the word is not in the cache:
- The entire block containing the word is loaded into the cache.
- The word is then delivered to the processor.

- These last two steps (loading and delivering) often happen concurrently.

In a common organization, the cache connects to the processor via data, control, and address lines. These lines also connect to buffers that interface with the system bus, which leads to main memory.

- During a **cache hit**, communication is direct between processor and cache; buffers are disabled, and no system bus traffic occurs. Think of this as a private conversation.
- During a **cache miss**, the address goes onto the system bus, and data returns through the data buffer to both the cache and the processor. This is like needing to ask someone outside the room for information.

Some architectures place the cache directly between the processor and main memory. In such cases, a miss involves reading into the cache first, then transferring to the processor.

3) Mapping Function

Since there are fewer cache lines than main memory blocks, a method is needed to decide which main memory block can occupy a cache line. This method is called the mapping function, and it determines the cache's organization.

Three mapping techniques exist:

- Direct Mapping
- Associative Mapping
- Set-Associative Mapping

1) Cache Addresses

Virtual memory is a feature supported by most processors, allowing programs to use logical addresses without being limited by the physical main memory size. When virtual memory is active, instructions use virtual addresses.

3.3.3 Elements of Cache Design

While many cache implementations exist, a few core design elements define and differentiate them. These elements are crucial for understanding various cache architectures. This section will cover the key parameters in cache design, with occasional references to their use in high-performance computing (HPC).

2) Cache Size

The ideal cache size balances cost and performance. It should be small enough that the average cost per bit approaches that of main memory, yet large enough for the average access time to approach that of the cache itself. Larger caches also require more complex addressing logic (more gates), which can increase overhead.

5) Write Policy

When a cache block needs to be replaced, its state determines the write operation.

- **Unmodified Block**: If the cache block hasn't been changed, it can be overwritten directly with new data.
- **Modified Block**: If the cache block has been written to, the modified data must first be written back to main memory before the new block can be loaded into the cache. This ensures data consistency.

Different write policies exist, each offering trade-offs between performance and cost.

4) Replacement Algorithms

When a new block of data needs to be loaded into a cache that is already full, an existing block must be removed (replaced).

- **Direct mapping** caches do not require a replacement algorithm, as each memory block has only one designated cache line.
- **Associative and set-associative** caches need a replacement algorithm to decide which block to evict.
- These algorithms must be implemented in hardware for speed.

Common Replacement Algorithms:

- **LRU (Least Recently Used)**: Considered the most effective. Evicts the block that has not been accessed for the longest time.
- **FIFO (First-In, First-Out)**: Evicts the oldest block in the cache.
- **LFU (Least Frequently Used)**: Evicts the block that has been accessed the fewest times.
- **Random replacement**: Evicts a randomly selected block.

7) Number of Caches

Modern systems often use multilevel caches, with at least one cache integrated directly onto the processor chip (on-chip cache).

Benefits of on-chip cache:

- Reduces processor's external bus activity.
- Speeds up execution and increases overall performance.

Design considerations for the number of caches:

- Number of cache levels.
- Whether caches are unified (handle both instructions and data) or split (separate caches for instructions and data).

6) Line size

Line size refers to the number of adjacent words retrieved from main memory and placed into the cache along with the desired word.

Initially, increasing the line size improves the hit ratio due to the principle of locality: data near a referenced word is likely to be needed soon. This brings more useful data into the cache.

However, as the line size grows very large, the hit ratio can decrease. This happens because the probability of using the newly fetched data becomes lower than the probability of needing to replace existing data that was brought in.

Analogy: Imagine a library. If you request one specific book (a word), and the librarian brings you that book plus a few related ones (adjacent words), you're likely to find them useful. But if they bring you an entire shelf of related books, some might be irrelevant, and you might have to put back useful books you already had to make space.